

REPUBLIQUE DU SENEGAL



Un peuple - un but - une foi

MINISTERE DE L'ENSEIGNEMENT SUPERIEUR DE LA RECHERCHE ET  
DE L'INNOVATION

Université Iba Der Thiam de Thiès



Licence 2 Mathématiques et Informatique

# RAPPORT DE PROJET

Annuaire d'une Fédération Sportive

**Présenté et réalisé par :**

Mamadou Malal Diallo

**Sous la direction de :**

Dr Abdoulaye Diallo

Année académique : 2025 - 2026

# Table des matières

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>Résumé</b>                                                     | <b>3</b>  |
| <b>1 Introduction</b>                                             | <b>4</b>  |
| <b>2 Bibliothèques utilisées</b>                                  | <b>5</b>  |
| 2.1 Windows.h . . . . .                                           | 5         |
| 2.2 conio.h . . . . .                                             | 5         |
| 2.3 stdlib.h . . . . .                                            | 5         |
| 2.4 string.h . . . . .                                            | 5         |
| 2.5 locale.h . . . . .                                            | 5         |
| 2.6 joueur.h . . . . .                                            | 5         |
| 2.7 club.h . . . . .                                              | 6         |
| 2.8 competition.h . . . . .                                       | 6         |
| 2.9 util.h . . . . .                                              | 6         |
| <b>3 Architecture</b>                                             | <b>7</b>  |
| 3.1 Vue d'ensemble . . . . .                                      | 7         |
| 3.2 Structures de données et organisation mémoire . . . . .       | 7         |
| 3.2.1 Structure Date . . . . .                                    | 7         |
| 3.2.2 Structure Adresse . . . . .                                 | 7         |
| 3.2.3 Structure Joueur . . . . .                                  | 7         |
| 3.2.4 Structure Club . . . . .                                    | 8         |
| 3.2.5 Structure Competition . . . . .                             | 8         |
| 3.2.6 Allocation dynamique . . . . .                              | 8         |
| 3.3 Gestion de la mémoire dynamique . . . . .                     | 9         |
| <b>4 Les fonctionnalités</b>                                      | <b>11</b> |
| 4.1 Menu joueurs . . . . .                                        | 11        |
| 4.1.1 Ajouter un joueur . . . . .                                 | 11        |
| 4.1.2 Afficher tous les joueurs . . . . .                         | 12        |
| 4.1.3 Rechercher un joueur . . . . .                              | 12        |
| 4.1.4 Supprimer un joueur . . . . .                               | 13        |
| 4.1.5 Mettre à jour un joueur . . . . .                           | 13        |
| 4.1.6 Trier les joueurs par id / nom . . . . .                    | 13        |
| 4.2 Menu clubs . . . . .                                          | 13        |
| 4.2.1 Ajouter un club . . . . .                                   | 13        |
| 4.2.2 Afficher tous les clubs . . . . .                           | 14        |
| 4.2.3 Rechercher un club . . . . .                                | 14        |
| 4.2.4 Supprimer un club . . . . .                                 | 14        |
| 4.2.5 Mettre à jour un club . . . . .                             | 15        |
| 4.2.6 Trier les clubs par id et trier les clubs par nom . . . . . | 15        |

|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| 4.2.7    | Ajouter un joueur dans un club . . . . .             | 15        |
| 4.2.8    | Afficher les joueurs d'un club . . . . .             | 16        |
| 4.3      | Menu compétitions . . . . .                          | 16        |
| 4.3.1    | Ajouter une compétition . . . . .                    | 16        |
| 4.3.2    | Ajouter des participants à une compétition . . . . . | 16        |
| 4.3.3    | Afficher les compétitions . . . . .                  | 17        |
| 4.3.4    | Rechercher une compétition . . . . .                 | 17        |
| 4.3.5    | Supprimer une compétition . . . . .                  | 17        |
| 4.3.6    | Supprimer un participant . . . . .                   | 17        |
| 4.3.7    | Mettre à jour une compétition . . . . .              | 18        |
| 4.3.8    | Trier les participants d'une compétition . . . . .   | 18        |
| 4.4      | Aller à l'accueil . . . . .                          | 18        |
| 4.5      | Quitter . . . . .                                    | 18        |
| <b>5</b> | <b>Etude de complexité</b>                           | <b>19</b> |
| 5.1      | Insertion . . . . .                                  | 19        |
| 5.2      | Recherche . . . . .                                  | 19        |
| 5.3      | Suppression . . . . .                                | 19        |
| 5.4      | Mise à jour . . . . .                                | 19        |
| 5.5      | Algorithme de tri . . . . .                          | 19        |
| 5.5.1    | Tri par sélection . . . . .                          | 19        |
| 5.5.2    | Tri à bulles . . . . .                               | 20        |
| 5.6      | Courbe de complexité . . . . .                       | 20        |
| 5.6.1    | Etude du tri par sélection . . . . .                 | 20        |
| 5.6.2    | Etude de la recherche séquentielle . . . . .         | 21        |
| <b>6</b> | <b>Conclusion et perspectives</b>                    | <b>22</b> |
| 6.1      | Bilan . . . . .                                      | 22        |
| 6.2      | Limites . . . . .                                    | 22        |
| 6.3      | Perspectives d'amélioration . . . . .                | 22        |

# Résumé

Dans le cadre du module Algorithmique et Structures de Données, nous avons développé une application de gestion d'une fédération sportive. Dans le cadre du module Algorithmique et Structures de Données, nous avons réalisé une application de gestion d'un annuaire de fédération sportive en langage C.

Cette application permet de gérer les clubs, les joueurs et les compétitions affiliés à une fédération sportive. Elle offre différentes fonctionnalités telles que l'ajout, la modification, la suppression, la recherche et le tri des données.

L'objectif principal est de mettre en pratique les notions de structures de données, de tableaux dynamiques, d'algorithmes de recherche et de tri, ainsi que la programmation modulaire en langage C.

# Chapitre 1

## Introduction

Le développement des systèmes informatiques a considérablement amélioré la gestion des organisations sportives. Les fédérations sportives doivent gérer un grand volume d'informations concernant les clubs, les joueurs et les compétitions.

La gestion manuelle de ces informations peut entraîner des erreurs, des pertes de données et un manque d'efficacité.

Pour répondre à ce besoin, nous avons développé une application permettant de centraliser et d'organiser toutes les informations relatives à une fédération sportive.

Les objectifs du projet sont :

- Gérer les clubs sportifs.
- Gérer les joueurs.
- Gérer les compétitions.
- Faciliter les recherches, les tri et les mises à jour.
- Mettre en pratique les concepts d'algorithmique et de structures de données.

# Chapitre 2

## Bibliothèques utilisées

Au cours de l'implémentation de l'application, plusieurs bibliothèques ont été utilisées :

### 2.1 Windows.h

Cette bibliothèque a servi à l'utilisation des fonctions propres à Windows, comme `SetConsoleOutputCP()` (pour permettre d'afficher les caractères UTF-8), `SetConsoleCP()` (pour permettre de lire les caractères UTF-8), `Sleep()`, ou modifier le titre de la console avec `system()` ou changer la couleur de police dans les menus.

### 2.2 conio.h

Cette bibliothèque a servi à bloquer la saisie utilisateur tant que la touche entrée n'est pas appuyé pour continuer.

### 2.3 stdlib.h

Cette bibliothèque a servi à gérer la mémoire et les fonctions système, par exemple `malloc`, `free`, `realloc`, `system`.

### 2.4 string.h

Cette bibliothèque a servi à manipuler les chaînes de caractères avec les fonctions `strcmp()` et `strcpy()`.

### 2.5 locale.h

Cette bibliothèque a servi à gérer la langue et l'encodage, notamment pour afficher correctement les caractères accentués avec la fonction `setlocale()`.

### 2.6 joueur.h

C'est une en-tête personnalisée pour les structures et fonctions liées aux joueurs.

## **2.7 club.h**

C'est une en-tête personnalisée pour les structures et fonctions liées aux clubs.

## **2.8 competition.h**

C'est une en-tête personnalisée pour les structures et fonctions liées aux compétitions.

## **2.9 util.h**

C'est une en-tête personnalisée pour les fonctions utilitaires du programme, comme effacer l'écran, nettoyer le tampon ou afficher des messages.

# Chapitre 3

## Architecture

### 3.1 Vue d'ensemble

L'application repose sur une architecture modulaire en couches :

- Couche métier : structures de données (joueurs, clubs, compétitions)
- Couche logique : modules spécialisés (joueur.c, club.c, competition.c)
- Couche présentation : interface utilisateur (main.c)
- Couche utilitaire : fonctions utilitaires (util.c)

### 3.2 Structures de données et organisation mémoire

#### 3.2.1 Structure Date

```
1      /*=====
2      Structure date
3      =====*/
4
5      typedef struct {
6          int jour; int mois; int annee;
7      } Date;
```

#### 3.2.2 Structure Adresse

```
1      typedef struct {
2          char siege[20], email[30], site_web[40], nom_stade[30],
3          adresse_stade[20];
4      } Adresse;
```

#### 3.2.3 Structure Joueur

```
1      /*=====
2      Structure joueur
3      =====*/
4      typedef struct {
5          char id[15];
6          char prenom[40], nom[40];
```



```

7         Date dateNaissance;
8         char nationalite[40];
9         char categorie[15];
10        char poste[30];
11        char clubActuel[40];
12
13    } Joueur;

```

### 3.2.4 Structure Club

```

1      /*=====
2      Structure club
3      =====*/
4      typedef struct {
5          char id[15];
6          char nom[40];
7          Date dateCreation;
8          Adresse adresse;
9          int points;
10         char proprietaire[20], manager[20];
11         int nb_pensionnaires, nb_pensionnaires_ajoutes;
12         Joueur *pensionnaires;
13     } Club;

```

Chaque club dispose d'un pointeur vers ses joueurs.

### 3.2.5 Structure Competition

```

1      /*=====
2      Structure competition
3      =====*/
4      typedef struct {
5          char id[15];
6          char nom[40];
7          char saison[10], type[40];
8          int nb_max_participants;
9          int nb_participants_ajoutes;
10         Club *classement;
11         Club *participants;
12     } Competition;

```

Chaque compétition dispose d'un pointeur vers ses participants (clubs).

### 3.2.6 Allocation dynamique

Au démarrage, l'application invite l'utilisateur à configurer le système. La configuration consiste à spécifier les capacités maximales des jeux de données (joueurs, clubs, compétitions). L'utilisateur devra saisir des valeurs entières. Tant que la valeur saisie n'est pas entière, un message d'erreur lui sera notifié et il sera encore invité à saisir une valeur entière. L'application alloue ainsi des tableaux dynamiques qui sont par la suite immuables.

```

1      joueurs = (Joueur *)malloc(nb_joueurs_max * sizeof(Joueur));
2      clubs = (Club *)malloc(nb_max_clubs * sizeof(Club));
3      competitions = (Compétition *)malloc(nb_max_compétitions *
4          sizeof(Compétition));

```

Après la configuration du système, l'application affiche le menu principal.

### 3.3 Gestion de la mémoire dynamique

Chaque ajout d'un objet (joueur, club, compétition) déclenche une vérification des capacités. Si la capacité maximale est atteinte alors une réallocation sera tentée par l'application, si elle échoue alors il ne sera pas possible d'effectuer l'ajout.

```

1      if (nb_joueurs_ajoutes >= nb_joueurs_max) {
2          Joueur *temp = realloc(joueurs, (nb_joueurs_max + 1) *
3              sizeof(Joueur));
4          if (temp == NULL) {
5              printf("
6
7              Nombre maximal de joueurs atteint (Echec de
8              l'allocation mémoire).\n");
9              return;
10         }
11         joueurs = temp;
12         nb_joueurs_max++;
13     }

```

```

1      if (nb_clubs_ajoutes >= nb_max_clubs)
2      {
3          Club *temp = realloc(clubs, (nb_max_clubs + 1) * sizeof
4              (Club));
5          if (temp == NULL){
6              printf("
7
8              Nombre maximal de clubs atteint (Echec de l'
9              allocation mémoire).\n");
10             return;
11         }
12         clubs = temp;
13         nb_max_clubs++;
14     }

```

```

1      if (nb_compétitions_ajoutes >= nb_max_compétitions) {
2          Compétition *temp = realloc(compétitions, (
3              nb_max_compétitions + 1) * sizeof(Compétition));
4          if (temp == NULL) {
5              printf("
6
7              Nombre maximal de compétitions atteint (
8              Echec de l'allocation mémoire).\n");
9              return;
10         }
11         competitions = temp;
12         nb_max_compétitions++;
13     }

```

```
6         }  
7         competitions = temp;  
8         nb_max_competitions++;  
9  
10    }
```

# Chapitre 4

## Les fonctionnalités

L'application propose plusieurs fonctionnalités réparties dans cinq (5) menus.

- 1- Menu joueurs
- 2- Menu clubs
- 3- Menu compétitions
- 4- Aller à l'accueil
- 5- QUITTER

### 4.1 Menu joueurs

Ce menu permet de gérer les joueurs. Ses fonctionnalités sont assurées par les fonctions implémentées dans le module joueur.c. Il propose 7 fonctionnalités.

#### 4.1.1 Ajouter un joueur

Cette option permet d'ajouter un joueur. Tout d'abord l'application vérifie que la taille maximale spécifiée lors de la configuration n'est pas atteinte. Si elle est atteinte l'ajout ne pourra pas se faire. Sinon l'utilisateur sera invité à remplir les champs du joueur. L'application s'assure que l'identifiant de chaque joueur soit unique pour éviter ainsi les doublons. Lorsque l'utilisateur termine de remplir les champs, le joueur sera ajouté dans le tableau joueurs.

```
1
2
3     if (nb_joueurs_ajoutes >= nb_joueurs_max) {
4         printf("Nombre maximal de joueurs atteint. Impossible d
5             'ajouter plus de joueurs.\n");
6         return;
7     }
8     int exist; char id[15];
9     do
10    {
11        exist = 0;
12        printf("Entrer l'identifiant du joueur : ");
13        scanf(" %[^\\n]", id);
14        for (int i = 0; i < nb_joueurs_ajoutes; i++)
15        {
16            if(strcmp(joueurs[i].id, id) == 0){
17                exist = 1;
```

```

17         printf("\n
18             Cet identifiant existe déjà.\n");
19         break;
20     }
21 } while (exist);
22
23 strcpy((joueurs+nb_joueurs_ajoutes)->id, id);
24 printf(" Entrer le prénom du joueur : ");
25 scanf(" %[^\n]", (joueurs+nb_joueurs_ajoutes)->prenom);
26 printf("Entrer le nom du joueur : ");
27 scanf(" %[^\n]", (joueurs+nb_joueurs_ajoutes)->nom);
28 printf("Entrer la date de naissance du joueur (jj mm aaaa) : ")
29 ;
30 scanf(" %d %d %d", &(joueurs+nb_joueurs_ajoutes)->dateNaissance
31 .jour,
32 &(joueurs+nb_joueurs_ajoutes)->dateNaissance.mois,
33 &(joueurs+nb_joueurs_ajoutes)->dateNaissance.annee);
34 printf("Entrer la nationalité du joueur : ");
35 scanf(" %[^\n]", (joueurs+nb_joueurs_ajoutes)->nationalite);
36 printf("Entrer la catégorie du joueur : ");
37 scanf(" %[^\n]", (joueurs+nb_joueurs_ajoutes)->categorie);
38 printf("Entrer le poste du joueur : ");
39 scanf(" %[^\n]", (joueurs+nb_joueurs_ajoutes)->poste);
40 printf("Entrer le club actuel du joueur : ");
41 scanf(" %[^\n]", (joueurs+nb_joueurs_ajoutes)->clubActuel);
42 puts("\n");
43 printf("%s %s a été ajouté avec succès !\n", (joueurs+
44     nb_joueurs_ajoutes)->prenom, (joueurs+nb_joueurs_ajoutes)->
45     nom);
46 nb_joueurs_ajoutes++;

```

### 4.1.2 Afficher tous les joueurs

Cette fonctionnalité permet d'afficher tous les joueurs enregistrés à l'aide de la fonction `liste_joueurs()`. Si aucun joueur n'est enregistré alors un message sera affiché pour notifier l'utilisateur.

```

1     if (nb_joueurs_ajoutes == 0) {
2         printf("
3             Liste vide. Aucun joueur à afficher.\n");
4         return;
5     }

```

S'il y a au moins un joueur enregistré alors ses informations seront affichées.

### 4.1.3 Rechercher un joueur

Cette fonctionnalité permet de rechercher un joueur à l'aide de la fonction `rechercher_joueur()`.

L'application vérifie en premier lieu si le tableau `joueurs` n'est pas vide, si c'est le cas un message d'erreur est affiché et l'application quitte immédiatement cette option. Autrement,

l'utilisateur saisit l'identifiant du joueur qu'il souhaite rechercher. Si un joueur avec cet identifiant est enregistré alors ses informations seront affichées. Sinon un message d'erreur sera notifié à l'utilisateur.

#### 4.1.4 Supprimer un joueur

La fonctionnalité de cette option est assurée par la fonction `supprimer_joueur()`.

L'application vérifie en premier lieu si le tableau `joueurs` n'est pas vide, si c'est le cas un message d'erreur est affiché et l'application quitte immédiatement cette option.

Autrement, l'utilisateur saisit l'identifiant du joueur qu'il souhaite supprimer. Si un joueur avec cet identifiant est enregistré alors il sera supprimé du tableau `joueurs`. Sinon un message d'erreur sera notifié à l'utilisateur. La suppression se fait par décalage à gauche. Le nombre de joueurs est ainsi décrémenté.

#### 4.1.5 Mettre à jour un joueur

Cette fonctionnalité permet de mettre à jour (modifier) un champ d'un joueur à l'aide de la fonction `mise_a_jour_joueur()`.

D'abord l'application s'assure qu'il y a au moins un joueur dans le tableau `joueurs` afin de tenter une modification et éviter d'utiliser le processeur inutilement.

S'il a un joueur dans le tableau `joueurs` alors l'utilisateur devra saisir l'identifiant du joueur qu'il souhaite modifier, puis l'application lancera une procédure de recherche du joueur ayant cet identifiant. Deux cas peuvent se présenter à l'issue de cette recherche.

- Le joueur est retrouvé : dans ce cas l'utilisateur sera invité à choisir le champ qu'il souhaite modifier. Et la modification se fera instantanément.
- Le joueur ayant l'identifiant saisi n'est pas retrouvé : Dans ce cas un message d'erreur sera affiché signalant qu'aucun joueur n'est enregistré dans le tableau `joueurs` avec l'identifiant saisi.

#### 4.1.6 Trier les joueurs par id / nom

Ces fonctionnalités permettent de trier les joueurs par id / nom à l'aide des fonctions `trier_joueurs_par_id()` et `trier_joueurs_par_nom()`.

S'il y a un joueur dans le tableau `joueurs` alors il sera trié selon le choix de l'utilisateur. Sinon un message d'erreur sera affiché signalant que le tableau `joueurs` est vide.

### 4.2 Menu clubs

Ce menu permet de gérer les clubs. Ses fonctionnalités sont assurées par les fonctions implémentées dans le module `club.c`. Il propose neuf (9) fonctionnalités.

#### 4.2.1 Ajouter un club

Cette fonctionnalité permet d'ajouter un club dans le tableau `clubs` à l'aide de la fonction `ajouter_club()`.

L'application vérifie en premier lieu si le tableau `clubs` est apte à enregistrer un nouveau club. Si le tableau est déjà plein alors un message d'erreur sera affiché signalant que l'enregistrement ne pourra pas s'effectuer.

Autrement, l'utilisateur sera invité à saisir l'identifiant du club. L'application lancera alors une recherche afin de s'assurer que ce club n'est pas déjà enregistré dans le tableau joueurs afin d'éviter des doublons. L'application s'assure ainsi que l'identifiant de chaque club est unique.

```
1      do
2      {
3          exist = 0;
4          printf("Entrer l'identifiant du club : ");
5          scanf(" %[^\\n]", id);
6          for (int i = 0; i < nb_clubs_ajoutes; i++)
7          {
8              if(strcmp(clubs[i].id, id) == 0){
9                  exist = 1;
10                 printf("Cet identifiant existe déjà.\\n"
11                     );
12                 break;
13             }
14         } while (exist);
```

Après la saisie d'un identifiant acceptable, l'utilisateur pourra remplir les champs restants.

#### 4.2.2 Afficher tous les clubs

Cette fonctionnalité permet d'afficher tous les clubs enregistrés dans le tableau clubs à l'aide de la fonction `afficher_club()`.

L'application affiche tous les joueurs enregistrés dans le tableau clubs si ce dernier n'est pas vide. Sinon il affichera un message d'erreur à la place.

#### 4.2.3 Rechercher un club

Cette fonctionnalité permet de rechercher tous les clubs enregistrés dans le tableau clubs à l'aide de la fonction `rechercher_club()`.

Si le tableau clubs n'est pas vide alors l'utilisateur sera invité à saisir l'identifiant du club qu'il souhaite rechercher.

```
1      printf("\\n Saisir l'identifiant du club recherché: ");
2      scanf(" %[^\\n]", id);
```

Puis l'application effectuera la recherche dans le tableau clubs. Si un club ayant cet identifiant est retrouvé alors ses informations seront affichées sinon un message d'erreur sera affiché à la place. Si le tableau clubs est vide alors un message d'erreur sera affiché et l'application quittera immédiatement la fonction.

```
1      if (nb_clubs_ajoutes == 0) {
2          printf("\\n Aucun club enregistré.\\n");
3          return;
4      }
```

#### 4.2.4 Supprimer un club

Cette fonctionnalité permet de supprimer un club du tableau clubs à l'aide de la fonction `supprimer_club()`.

Si le tableau clubs est vide alors l'application affiche un message d'erreur avant de quitter la fonction. Sinon l'application invitera l'utilisateur à saisir l'identifiant du club qu'il souhaite supprimer. Le club ayant cet identifiant sera alors supprimé s'il est présent dans le tableau clubs ou bien un message d'erreur sera affiché à la place s'il y est absent.

#### 4.2.5 Mettre à jour un club

Cette fonctionnalité permet de mettre à jour un champ d'un club enregistré dans le tableau club à l'aide de la fonction `mise_a_jour_club()`.

Si le tableau clubs est vide alors l'application affiche un message d'erreur avant de quitter la fonction. Sinon l'application invitera l'utilisateur à saisir l'identifiant du club qu'il souhaite modifier puis l'application lancera une procédure de recherche du club ayant cet identifiant. Deux cas peuvent se présenter à l'issue de cette recherche.

- Le club est retrouvé : dans ce cas l'utilisateur sera invité à choisir le champ qu'il souhaite modifié. Et la modification se fera instantanément.
- Le club ayant l'identifiant saisi n'est pas retrouvé : Dans ce cas un message d'erreur sera affiché signalant qu'aucun club n'est enregistré dans le tableau clubs avec l'identifiant saisi.

#### 4.2.6 Trier les clubs par id et trier les clubs par nom

Ces fonctionnalités permettent de trier le tableau clubs par id à l'aide de la fonction `trier_clubs_par_id()` (tri par sélection) ou bien par nom à l'aide de la fonction `trier_clubs_par_nom()` (tri à bulles).

Si le tableau clubs est vide alors l'application affiche un message d'erreur avant de quitter la fonction. Sinon le tableau sera trié selon le choix de l'utilisateur.

#### 4.2.7 Ajouter un joueur dans un club

Cette fonctionnalité permet d'ajouter un joueur dans un club enregistré dans le tableau clubs à l'aide de la fonction `ajouter_joueur_club()`.

Si le tableau clubs ou le tableau joueurs est vide alors l'application affiche un message d'erreur avant de quitter la fonction.

```
1      if(nb_clubs_ajoutes == 0){
2          printf("Aucun club enregistré\n");
3          return;
4      }
5
6      if(nb_joueurs_ajoutes == 0){
7          printf("Aucun joueur enregistré\n");
8          return;
9      }
```

Si les conditions mentionnées ci-haut sont fausses alors l'utilisateur devra saisir l'identifiant du club auquel il souhaite ajouter un joueur. Après cette saisie, on distinguera deux cas :

- Aucun club n'est enregistré dans le tableau clubs avec l'identifiant saisi. Dans ce cas l'application affiche un message d'erreur avant de quitter la fonction.

```
1  if(!club_trouve){
2      printf("\n Aucun club enregistré avec cet identifiant\n",
3          id_club);
4  }
```



- Un club ayant cet identifiant est présent dans le tableau clubs. Dans ce cas l'utilisateur sera invité à saisir l'identifiant du joueur qu'il souhaite ajouter dans ce club. Après cette saisie, on distinguera trois résultats possibles :
  - Le joueur ayant cet identifiant est déjà enregistré dans ce club. Alors l'application notifie l'utilisateur et quitte la fonction.
  - Le joueur ayant cet identifiant est retrouvé dans le tableau joueurs mais n'est pas un joueur de ce club, dans ce cas le joueur sera automatiquement enregistré dans le tableau pensionnaires de ce club.
  - Aucun joueur avec cet identifiant n'est enregistré dans le tableau joueurs, dans ce cas l'application affiche un message d'erreur avant de quitter la fonction.

**Remarque :** Le tableau pensionnaires est un tableau dynamique réalloué à chaque fois qu'il sera possible d'ajouter un joueur dans un club. Ceci permet d'éviter un gaspillage de l'espace mémoire du système.

#### 4.2.8 Afficher les joueurs d'un club

Cette fonctionnalité permet d'afficher tous les joueurs d'un club enregistré dans le tableau clubs à l'aide de la fonction `afficher_joueur_du_club()`.

L'utilisateur sera invité à saisir l'identifiant du club dont il souhaite afficher les joueurs. Si ce club est présent dans le tableau clubs alors une liste de ses joueurs sera affichée (si le tableau pensionnaires de ce club n'est pas vide) sinon un message d'erreur sera affiché à la place.

### 4.3 Menu compétitions

Ce menu permet de gérer les compétitions. Ses fonctionnalités sont assurées par les fonctions implémentées dans le module `competition.c`. Il propose huit (8) fonctionnalités.

#### 4.3.1 Ajouter une compétition

Cette fonctionnalité permet d'ajouter une compétition dans le tableau competitions à l'aide de la fonction `ajouter_competition()`.

L'application vérifie en premier lieu si le tableau competitions est apte à enregistrer une nouvelle compétition. Si le tableau est déjà plein alors un message d'erreur sera affiché signalant que l'enregistrement ne pourra pas s'effectuer.

Autrement, l'utilisateur sera invité à saisir un identifiant pour la compétition. L'application vérifie alors si l'identifiant saisi n'est pas déjà attribué à une compétition dans le tableau competitions. Si une compétition ayant cet identifiant est y déjà présent, alors l'utilisateur devra saisir un nouveau identifiant différent. Cela garantit l'unicité de l'identifiant de chaque compétition. Après cette opération, l'utilisateur pourra saisir les autres champs qui lui seront demandés.

A la fin des saisies, la compétition sera enregistré dans le tableau competitions.

#### 4.3.2 Ajouter des participants à une compétition

Cette fonctionnalité permet d'ajouter un participant à une compétition à l'aide de la fonction `ajouter_participants()`.

L'application invite l'utilisateur à spécifier l'identifiant de la compétition dans laquelle il souhaite ajouter un participant (club). On distinguera alors deux cas :

- La compétition est enregistrée dans le tableau competition. Dans ce cas l'utilisateur sera invité à saisir l'identifiant du club qu'il souhaite ajouter dans cette compétition. Si ce

club existe dans le tableau clubs alors il sera automatiquement ajouté dans le tableau participants de cette compétition s'il n'y est pas déjà présent. Sinon un message d'erreur sera affiché signalant l'inexistence d'un club ayant l'identifiant saisi pour le club.

- La compétition n'est pas enregistrée dans le tableau competition. Dans ce cas un message s'affichera signalant l'inexistence d'une compétition ayant l'identifiant saisi pour la compétition.

**Remarque :** Un même club ne peut être ajouté plus d'une fois dans une même compétition. Cela évite les doublons et respecte les critères d'une compétition sportive.

### 4.3.3 Afficher les compétitions

Cette fonctionnalité permet d'afficher toutes les compétitions enregistrées dans le tableau competitions à l'aide de la fonction `afficher_competition()`.

Si le tableau competitions est vide alors un message d'erreur s'affichera notifiant sur l'état de vacuité du tableau. Autrement, une liste d'informations sur toutes les compétitions qui y sont enregistrées sera affichée.

### 4.3.4 Rechercher une compétition

Cette fonctionnalité permet de rechercher une compétition enregistrée dans le tableau competitions, à l'aide de la fonction `rechercher_competition()`.

Si le tableau competitions est vide alors un message d'erreur sera affiché signalant l'état de vacuité du tableau. Autrement, l'utilisateur devra saisir l'identifiant de la compétition qu'il souhaite rechercher. On distinguera alors deux cas :

- La compétition recherchée est présente dans le tableau competitions, dans ce cas, ses informations seront affichées.
- La compétition recherchée est absente dans le tableau competitions, dans ce cas, un message d'erreur sera affiché pour notifier son absence.

### 4.3.5 Supprimer une compétition

Cette fonctionnalité permet de supprimer une compétition du tableau competitions, à l'aide de la fonction `supprimer_competition()`.

Si le tableau competitions est vide alors un message d'erreur sera affiché signalant l'état de vacuité du tableau. Autrement, l'utilisateur devra saisir l'identifiant de la compétition qu'il souhaite supprimer. On distinguera alors deux cas :

- Cette compétition est présente dans le tableau competitions, dans ce cas, elle sera supprimée du tableau.
- Cette compétition n'est pas présente dans le tableau competitions, dans ce cas, un message d'erreur sera affiché pour notifier son absence.

### 4.3.6 Supprimer un participant

Cette fonctionnalité permet de supprimer un participant (club) d'une compétition, à l'aide de la fonction `supprimer_participant()`.

Si le tableau competitions est vide alors un message d'erreur sera affiché signalant l'état de vacuité du tableau. Autrement, l'utilisateur devra saisir l'identifiant de la compétition où est enregistré le participant qu'il souhaite supprimer. On distinguera alors deux cas :

- Cette compétition est présente dans le tableau competitions, dans ce cas, il devra saisir l'identifiant du participant et s'il est présent dans le tableau participants de cette

compétition alors il sera supprimé de ce tableau sinon un message d'erreur sera affiché notifiant son absence.

- Cette compétition n'est pas présente dans le tableau competitions, dans ce cas, un message d'erreur sera affiché pour notifier son absence.

**Remarque :** Si la compétition et le participant sont trouvés, alors le nombre de participants à cette compétition sera décrémenté après la suppression.

#### 4.3.7 Mettre à jour une compétition

Cette fonctionnalité permet de modifier un ou des champs d'une compétition à l'aide de la fonction `mise_a_jour_competition()`.

#### 4.3.8 Trier les participants d'une compétition

Cette fonctionnalité permet de trier les participants d'une compétition selon leur nombre de points dans l'ordre décroissant, à l'aide de la fonction `trier_participants()`.

### 4.4 Aller à l'accueil

Ce menu affiche la page de présentation de l'application. Il donne des informations sur le développeur de l'application et le nom du projet ayant donné naissance à l'application.

### 4.5 Quitter

Ce menu permet de quitter l'application définitivement.

**Remarque :** Une fois l'application quittée, toutes les données seront perdues.

# Chapitre 5

## Etude de complexité

### 5.1 Insertion

L'insertion d'un enregistrement (joueur, club, compétition) suit le même schéma :

1. Vérification que la capacité maximale n'est pas atteinte ;
2. Vérification de l'unicité de l'identifiant par recherche séquentielle ;
3. Saisie interactive des champs via scanf ;
4. Stockage à l'indice nb\_ajoutes et incrémentation du compteur.

La complexité est en  $O(n)$  pour le contrôle d'unicité (recherche séquentielle).

### 5.2 Recherche

La recherche est effectuée par parcours séquentiel du tableau, avec comparaison des identifiants via strcmp. Si l'élément recherché est à la première position du tableau alors la complexité est  $O(1)$  (dans le meilleur cas), s'il se trouve au milieu ou à la fin du tableau ou encore qu'il n'est pas dans le tableau alors la complexité est  $O(n)$  (dans le moyen et pire cas).

### 5.3 Suppression

La suppression est réalisée par **décalage du tableau** : tous les éléments situés après la position supprimée sont décalés d'un rang vers la gauche.  
la complexité est  $O(n)$  pour cette opération.

### 5.4 Mise à jour

La mise à jour localise d'abord l'enregistrement par recherche séquentielle, puis propose un sous-menu permettant de modifier le champ souhaité. La complexité La complexité dépend alors de la recherche séquentielle.

### 5.5 Algorithme de tri

#### 5.5.1 Tri par sélection

Le tri par sélection parcourt le tableau pour trouver l'élément minimum et l'échanger avec la position courante.

Complexité :  $O(n^2)$  dans tous les cas.

### 5.5.2 Tri à bulles

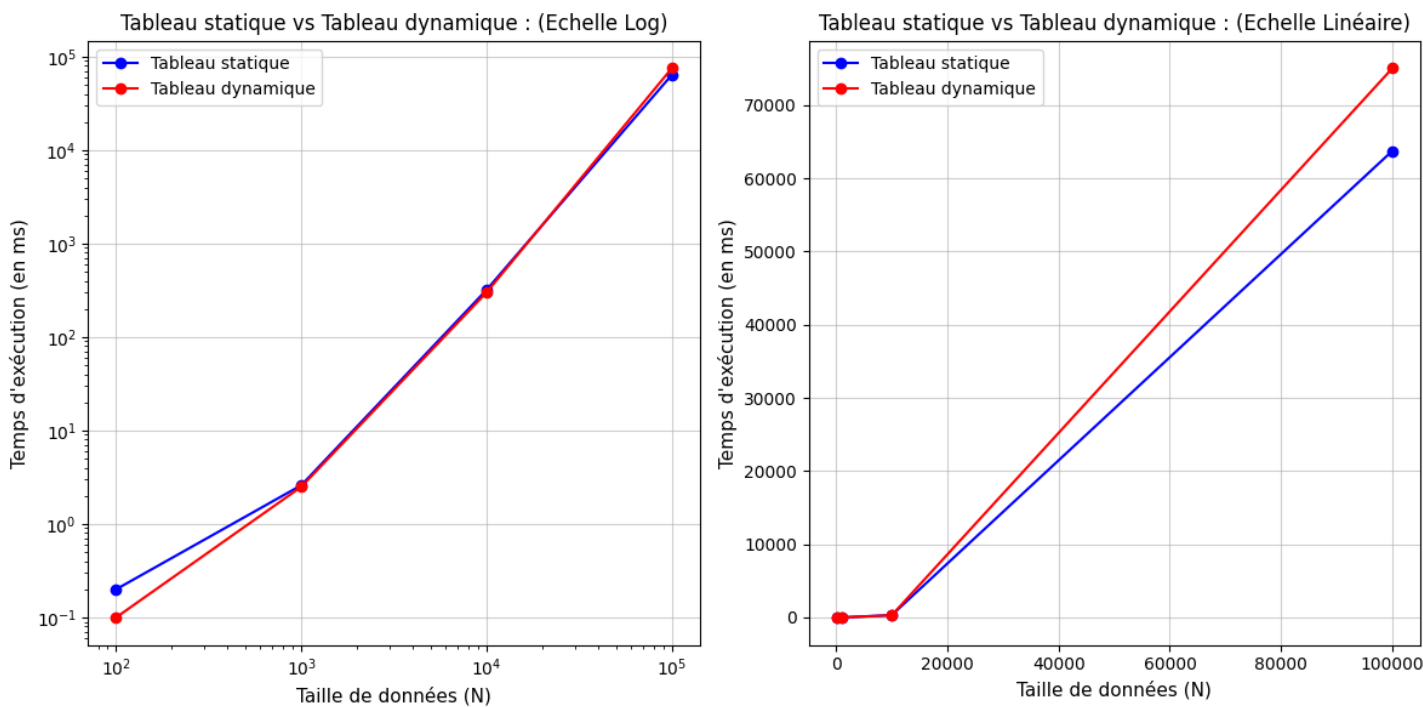
Le tri à bulles compare des paires adjacentes et les échange si elles sont dans le mauvais ordre. Complexité :  $O(n^2)$  dans tous les cas.

## 5.6 Courbe de complexité

Les fonctions de recherche et de tri ont été testées sur des tableaux statiques et des tableaux dynamiques afin de faire une étude comparative.

### 5.6.1 Etude du tri par sélection

Des expériences ont été réalisées dans des tableaux dynamiques et statiques manipulant les mêmes objets pour savoir lequel des deux types de tableaux est le plus favorable au tri par sélection. Le graphique ci-dessous montrent les résultats obtenues à l'issue de ces expériences :

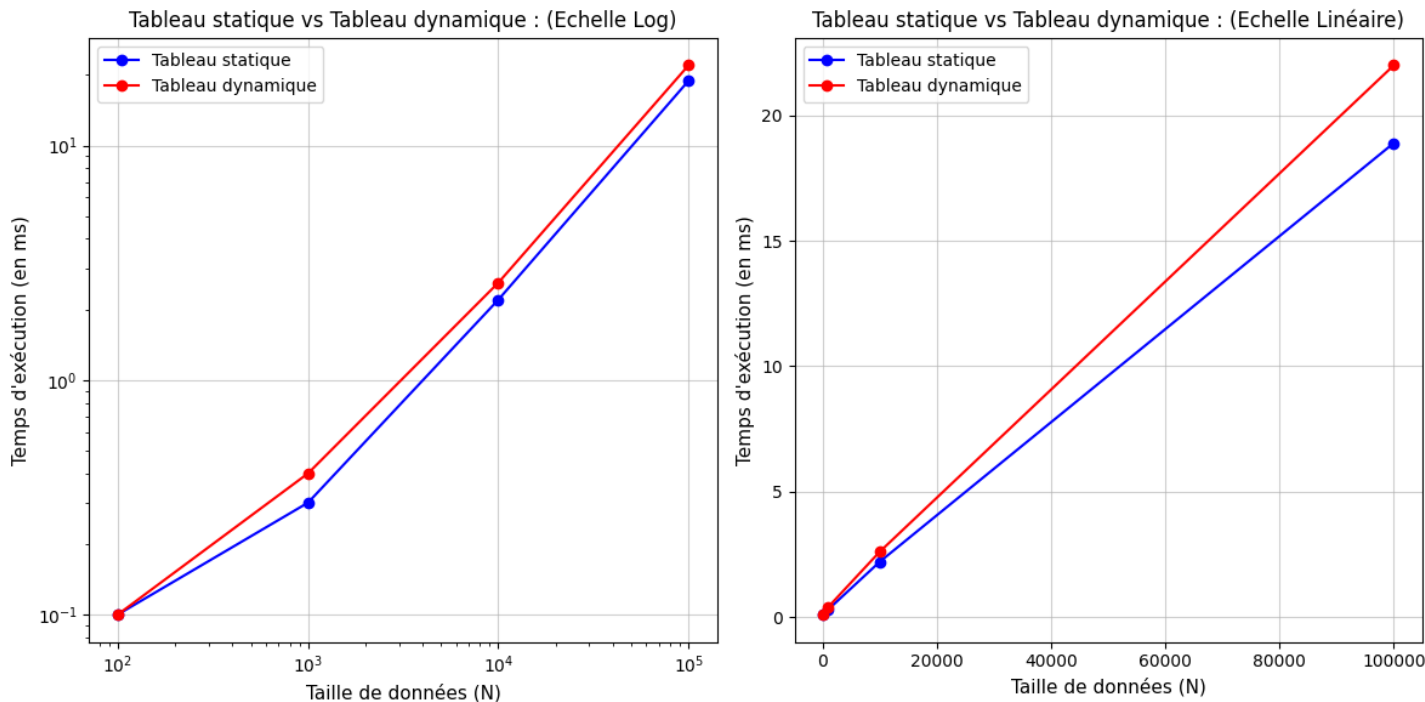


Soit  $N$  la taille des données.

D'après ce graphique, on constate que pour  $N \in [10^2, 10^3]$ , le tri par sélection dans un tableau dynamique est plus rapide que celui dans un tableau statique. Pour  $N \in [10^3, 10^4]$ , le coût du tri semble le même dans les deux tableaux. Et pour  $N \in [10^4, 10^5]$ , le tri est plus rapide dans un tableau statique que dans un tableau dynamique. D'après ces observations, pour une quantité de données  $N \leq 10^4$  l'utilisation d'un tableau dynamique est préférable pour le tri. Cependant pour une quantité de données  $N > 10^4$ , l'utilisation d'un tableau statique est le plus recommandé.

### 5.6.2 Etude de la recherche séquentielle

Des expériences ont été réalisées dans des tableaux dynamiques et statiques manipulant les mêmes objets pour savoir lequel des deux types de tableaux est le plus favorable à la recherche séquentielle. Le graphique ci-dessous montrent les résultats obtenues à l'issue de ces expériences :



Soit  $N$  la taille des données.

D'après ce graphique, pour  $N \in [100, 100000]$ , la recherche est plus rapide dans un tableau statique que dans un tableau dynamique.

Donc pour la recherche séquentielle, il est préférable d'opter pour des tableaux statiques.

# Chapitre 6

## Conclusion et perspectives

### 6.1 Bilan

Ce projet a permis de mettre en pratique les concepts fondamentaux de l’algorithmique et des structures de données en langage C :

- La conception de type enregistrements (struct) complexes et imbriqués avec des relations inter-entités réalisées par des pointeurs ;
- L’implémentation des opérations classiques (insertion, suppression, recherche, tri) sur des tableaux alloués dynamiquement ;
- L’analyse de complexité asymptotique des algorithmes mis en œuvre ;
- La structuration du code en modules indépendants avec un Makefile fonctionnel ;
- La production d’une interface utilisateur interactive en console.

Le domaine de l’annuaire de fédération sportive s’est avéré particulièrement adapté à ce projet, car il implique naturellement des entités hiérarchiques (fédération → compétitions → clubs → joueurs), favorisant l’utilisation de pointeurs et de tableaux dynamiques.

### 6.2 Limites

- L’utilisation de **Windows.h**, **conio.h** et **Sleep()** rend le code non portable sur Linux/macOS ;
- Toutes les données sont perdues à la fermeture de l’application ;
- L’utilisateur revient toujours au menu principal après une opération ;
- Les caractères accentués ne sont pas acceptés dans les tableaux de caractères des enregistrements (Joueur, Club et Compétition).

### 6.3 Perspectives d’amélioration

- Ajouter une fonctionnalité permettant d’enregistrer les données dans un fichier et de les restaurer si besoin.
- Permettre la portabilité de l’application sur Linux/macOS ;
- Liste doublement chaînée : alternative aux tableaux pour les insertions et suppressions fréquentes en milieu de collection ( $O(1)$  après localisation) ;
- Forcer l’application à accepter les caractères accentués dans les tableaux de caractères des enregistrements (Joueur, Club et Compétition).

Le lien suivant permet d’accéder au dépôt GitHub du projet :

[https://github.com/Malaldo/Annuaire\\_federation\\_sportive](https://github.com/Malaldo/Annuaire_federation_sportive)